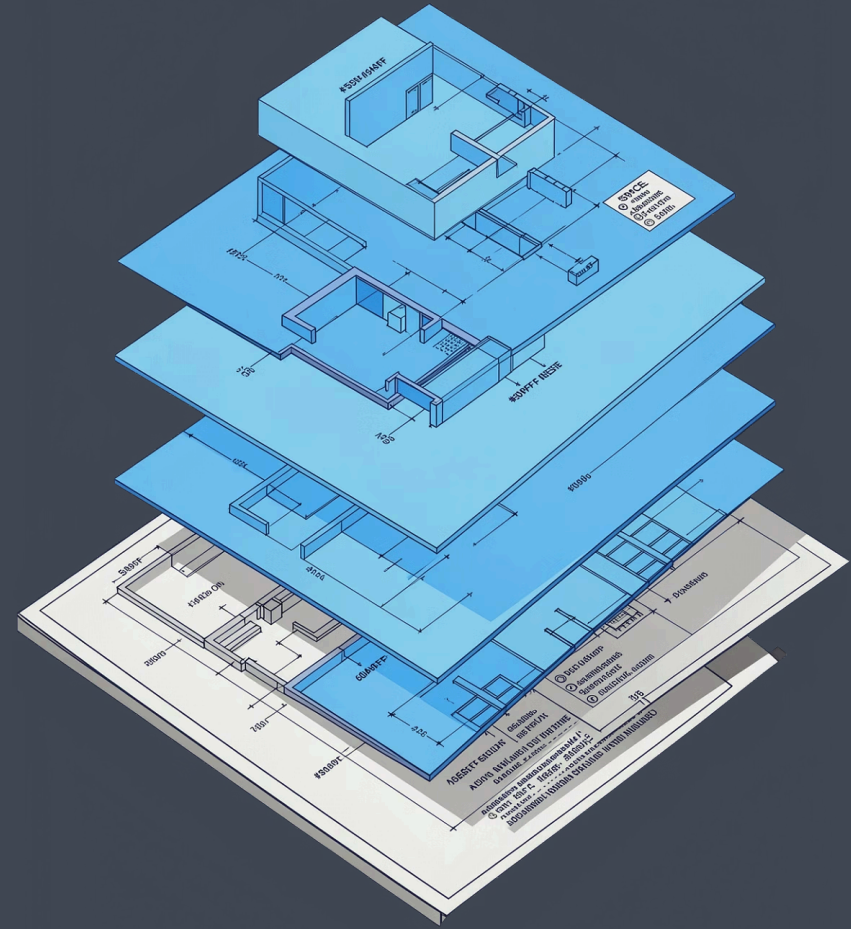


Java & Python

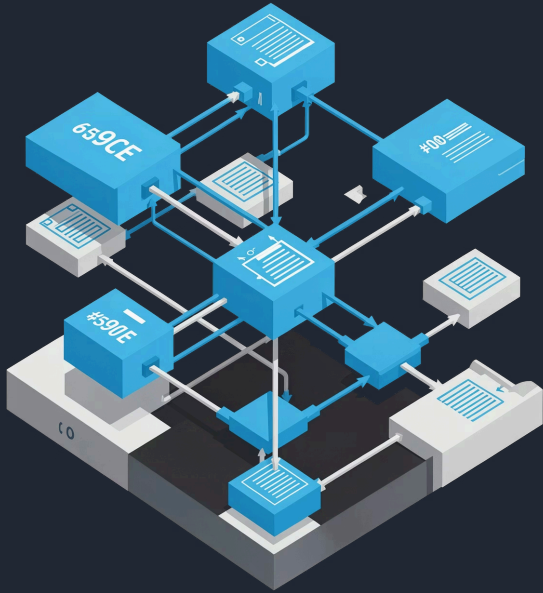
Full Stack Web : Comprendre l'architecture moderne

Une introduction structurée pour ingénieurs Java expérimentés — sans repartir de zéro.

copyright © Katy Saintin



Qu'est-ce que le Full Stack ?



Full Stack = travailler sur plusieurs couches d'une application Web. Une application Web n'est jamais isolée — elle est composée de parties qui communiquent entre elles.

Le flux d'une requête Web

01

L'utilisateur clique dans le navigateur

02

Le **Front End** envoie une demande

03

L'**API** transmet la demande

04

Le **Back End** traite et renvoie les données

Front End et Back End : l'analogie du restaurant

Au restaurant

Front End

La salle, le serveur, l'interface avec le client

API

Le bon de commande — la communication entre salle et cuisine

Back End

La cuisine, les traitements internes, la logique

En informatique

Front End

Navigateur Web, interface utilisateur (IHM)

API

Protocole HTTP/REST pour communiquer

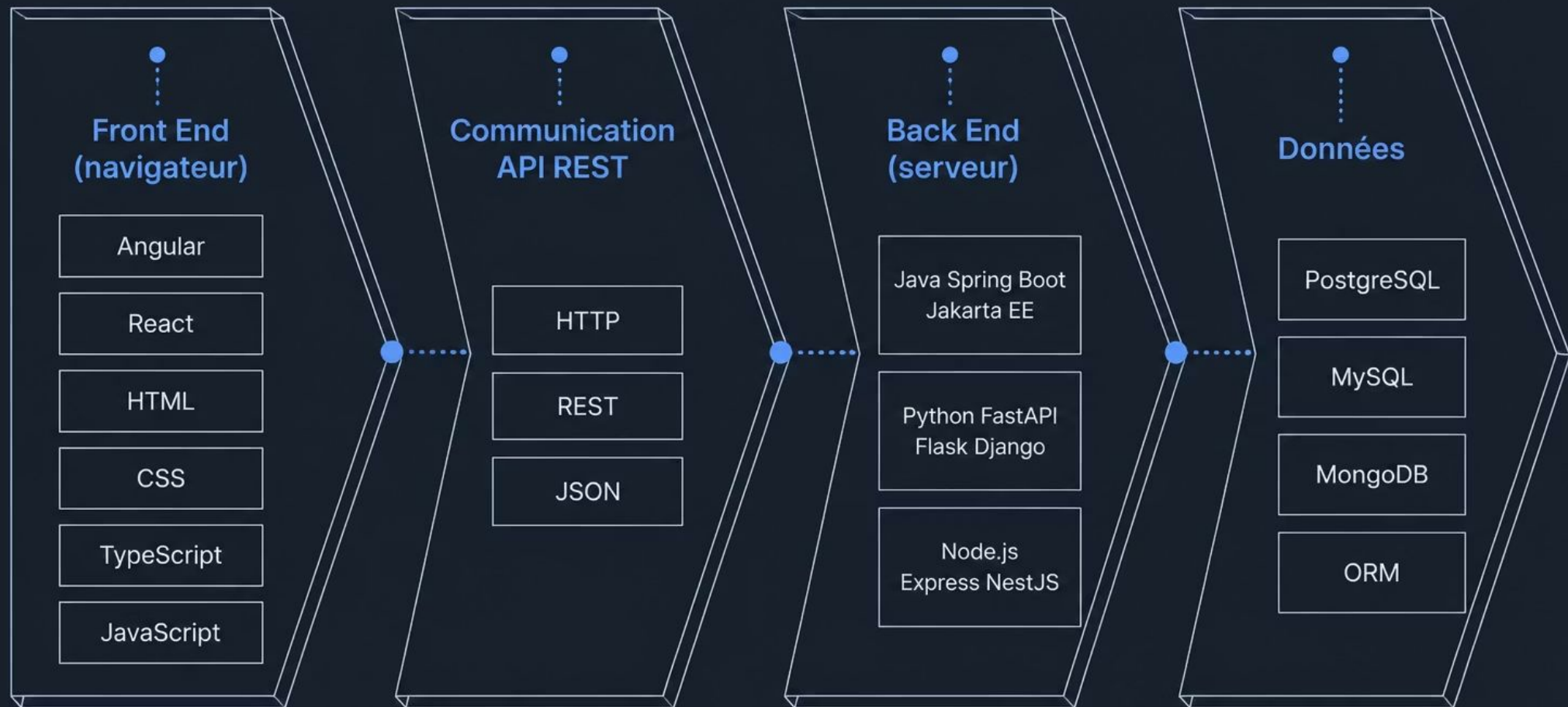
Back End

Serveur, traitements métier, logique applicative

 Front End et Back End sont des **zones de l'architecture**, pas des langages de programmation.



Où placer les technologies Web ?



Chaque technologie occupe une zone précise — apprendre le Full Stack, c'est comprendre ces rôles.

Java Web : partir de vos connaissances

Vous connaissez déjà Java, la POO et les design patterns. Voici comment ils s'appliquent au Web.



JVM

Machine virtuelle Java
— exécute le bytecode

Maven / Gradle

Build et gestion des
dépendances
(successeurs d'Ant)

Spring Boot

Créer des API REST
rapidement — comme
Swing pour le client
lourd

Jakarta EE

Standard Java entreprise (ex-Java
EE)

JPA / Hibernate

ORM : mapper les objets Java aux
tables SQL

Python Web : l'écosystème alternatif

Python offre plusieurs chemins pour le Web. Contrairement à Java, il n'y a pas un seul standard.



Poetry / uv / pip

Gestion des dépendances et de l'environnement virtuel



Flask

Microframework léger et minimaliste



FastAPI

Framework moderne, optimisé pour les API REST



Django

Framework complet : ORM, admin, authentification intégrés



SQLAlchemy

ORM Python — équivalent de Hibernate



Streamlit

IHM Web rapide, sans HTML ni CSS



Différence clé : Python n'a pas de machine virtuelle comme la JVM. Le code s'exécute directement.



Front End : Angular, React, TypeScript

Ces technologies vivent dans le **navigateur**. Elles créent l'interface que l'utilisateur voit et manipule.

1

HTML

Structure — balises et éléments

2

CSS

Présentation — couleurs, mise en page

3

TypeScript

JavaScript avec typage statique — comme Java pour le Web

4

Angular / React

Frameworks IHM — comme Swing ou JavaFX, dans le navigateur

API REST et JSON : le langage commun

Le Front End et le Back End communiquent via une **API REST** en **JSON**.

Exemple concret

GET /api/users/42

Réponse JSON :

```
{  
  "id": 42,  
  "name": "Alice",  
  "email": "alice@example.com"  
}
```

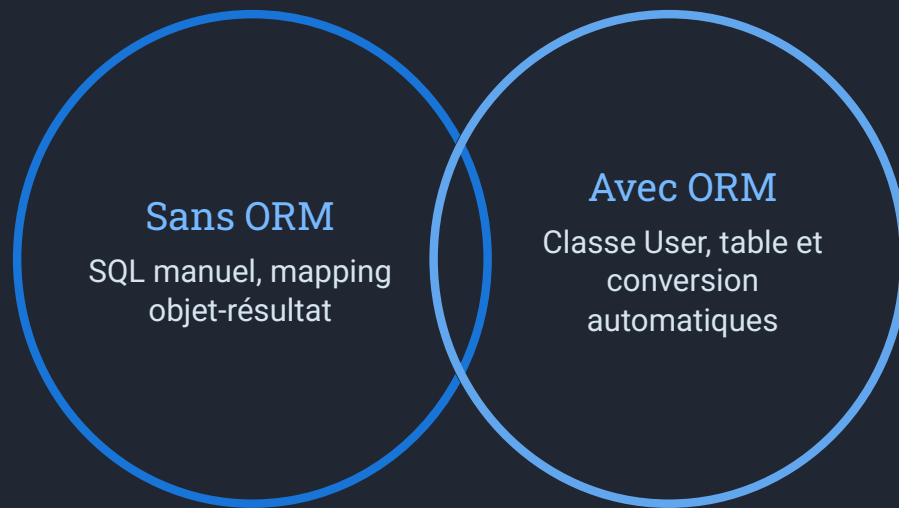


Les concepts clés

- **HTTP** — protocole de communication sur le Web
- **REST** — style architectural : GET, POST, PUT, DELETE
- **JSON** — format texte léger et lisible pour échanger des données
- **API** — contrat entre Front End et Back End

❏ REST n'est ni Front End ni Back End. C'est le **pont** entre les deux.

Données et ORM : mapper les objets à la base



Les ORM courants

Bases de données supportées

- Relationnelles : PostgreSQL, MySQL, SQLite
- Non-relationnelles : MongoDB, Redis

L'ORM automatise la conversion objet-relationnel — c'est un design pattern que vous connaissez déjà.

Java

JPA (standard) + **Hibernate** (implémentation)

Python

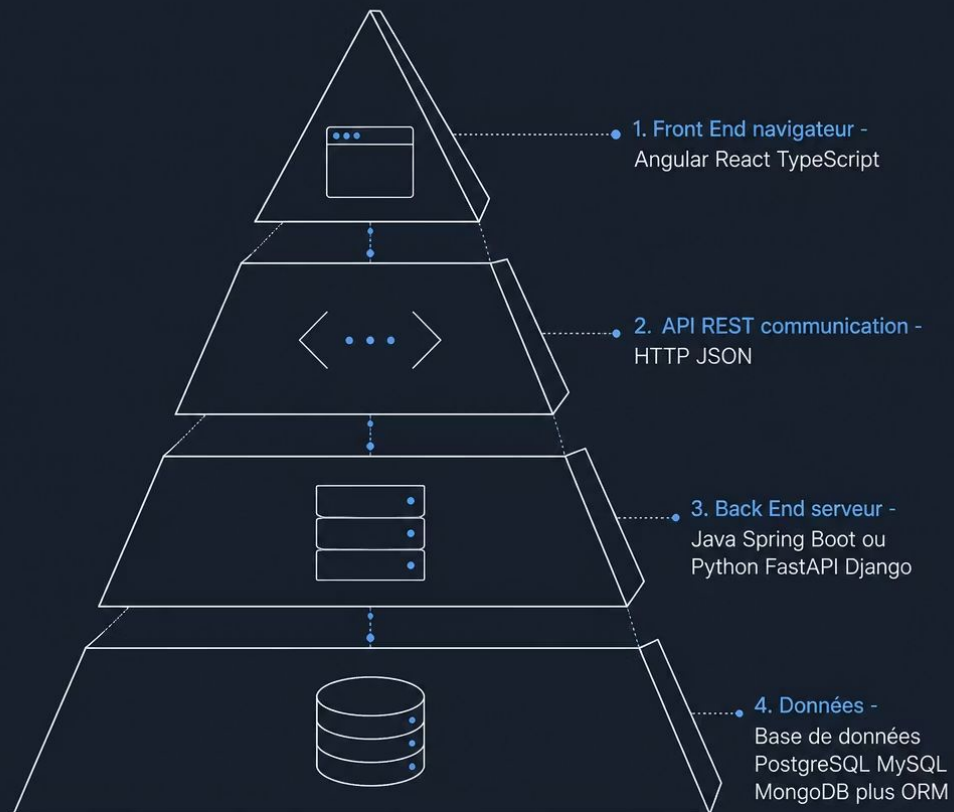
SQLAlchemy (indépendant) ou **Django ORM** (intégré)

Tableau comparatif : Java vs Python

Java et Python sont comparables côté **Back End**. Le Front End (Angular, React) est **indépendant** du langage serveur.

Besoin	Java	Python
Front End	Angular, React, TypeScript	Angular, React, TypeScript (<i>identique</i>)
API REST	Spring Boot, Jakarta REST	FastAPI, Flask, Django REST
Framework Web	Spring Boot, Jakarta EE	Django, Flask, FastAPI
Build / Dépendances	Maven, Gradle	Poetry, uv, pip
ORM	JPA, Hibernate	SQLAlchemy, Django ORM
Runtime	JVM (bytecode compilé)	Python interprété (pas de VM)
IHM Web rapide	—	Streamlit

La carte mentale à retenir



Les 10 mots-clés à retenir

« Je n'ai pas besoin de connaître tous les frameworks pour comprendre une architecture Full Stack. Je dois d'abord savoir quel **rôle** joue chaque technologie. »

Vous maîtrisez déjà la POO et les design patterns. Le Web, c'est juste une autre façon d'organiser ces concepts.

Full Stack

Front End

Back End

API

REST

JSON

Angular

Spring Boot

FastAPI

Database